

Dynamic Load Balancing and Persistent Chat

CSD Course — Lab Assignment 6

Student Name	Rahul Raj
Roll Number	12341680
Date	04 September 2026
Public URL	http://10.1.75.53:3269 (dynamic load balancer on sys1; dashboard at /lb/)
Source code	https://github.com/Rahul5977/csd-lab6-dynamic-lb-persistent-chat

1. Assigned Systems and Ports

Role	Alias	SSH	Container IP	Service port(s)	External port
Load balancer + database service	stu78_sys1	ssh -p 2269 student@10.1.75.53	172.17.0.70	LB 3000 · SQLite DB service 5270	3269 → LB (the only URL clients use)
Backend A	stu78_sys2	ssh -p 2270 student@10.1.75.53	172.17.0.71	app 3000	3270 (direct, kept for the previous assignment)
Backend B	stu78_sys3	ssh -p 2271 student@10.1.75.53	172.17.0.72	app 3000	3271 (direct, previous assignment)
Backend C	stu78_sys4	ssh -p 2272 student@10.1.75.53	172.17.0.73	app 3001	— (port 3000 / external 3272 belongs to my course project)

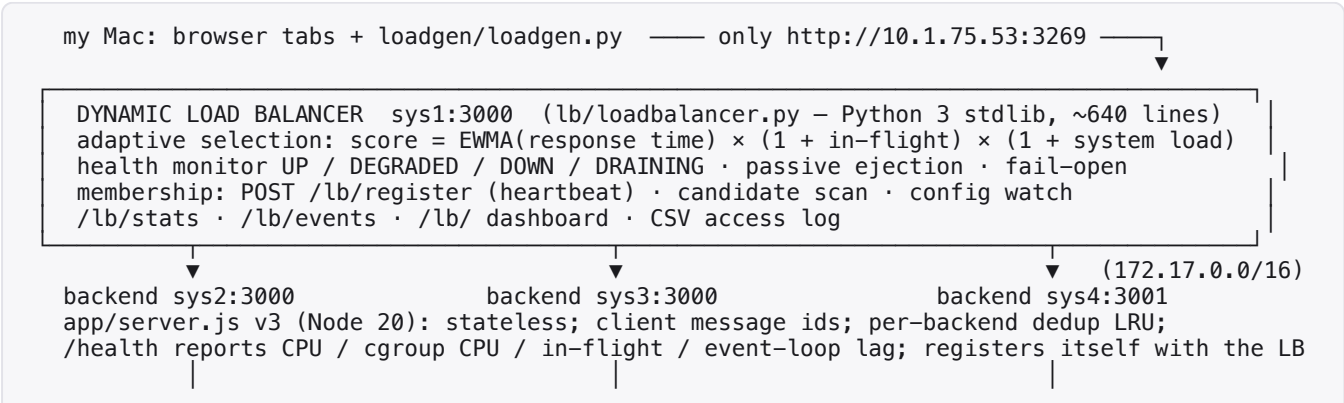
The four systems are Ubuntu 24.04 containers on one shared lab host, each limited by cgroup to **exactly one CPU core**. The lab NAT forwards only the first external port of each system to container port 3000. Clients — the browser and the load generator on my Mac — talk only to `10.1.75.53:3269`; the load balancer reaches the backends over the private bridge network, so a backend does not need a public port at all (which is why sys4's backend can live on 3001). The load generator runs on my local machine.

2. Objective

Extend the previous secure group-chat application and deploy its backend on at least three allotted systems behind a load balancer. The load balancer must select a backend **dynamically** using response time and/or current system load, monitor backend health and stop sending to unavailable backends, and **automatically detect backends started while the application is running**. All backends must share **persistent** chat data, every message must have a **unique id**, and duplicate insertion must be prevented on retries/reconnections. Generate increasing load, record response time, throughput, successful/failed requests and the number of active backends, and analyse the results.

Everything above is implemented, deployed on the allotted systems, measured, and demonstrated in this report. The base is my Assignment-5 system (Python load balancer, Node.js chat backend with the Assignment-4 end-to-end encryption); Assignment 6 adds a second-generation balancer, a real database, and idempotent messaging. The previous assignment keeps working: its files and directories on the lab systems are untouched and `scripts/rollback_lab5.sh` restores its processes in twenty seconds.

3. Architecture



```
DATABASE SERVICE sys1:5270 (app/db_service.js – Node 22, node:sqlite, WAL)
users · sessions · rooms · messages(id PRIMARY KEY, UNIQUE(room,seq)) · dedup_log
POST /messages is idempotent (INSERT ... ON CONFLICT(id) DO NOTHING in one transaction)
WebSocket firehose → every backend delivers new messages to its own connected clients
```

Request lifecycle. A request arrives at the balancer, which picks a backend with the adaptive algorithm (§4), adds `X-Forwarded-For` / `X-Real-IP` and proxies it over a pooled keep-alive connection. The backend validates the session (30 s local cache, source of truth in the database), performs the operation against the database service and answers with `X-Backend-Id` (which instance served it) — the balancer adds `X-LB-Active-Backends` (how many backends were routable at that moment); the load generator records both on every request. A new message is appended once to SQLite; the database broadcasts it on a WebSocket firehose and every backend pushes it to its own connected browsers, so three stateless backends behave as one application. Browser WebSockets are pinned to a backend by IP-hash and tunnelled byte-for-byte.

What is inherited from Assignments 4/5 unchanged: script password hashing with constant-time compare and login rate limiting; end-to-end encrypted "locked" rooms (PBKDF2-HMAC-SHA-256 with 250 000 iterations → AES-256-GCM, fresh IV per message, AAD binding room and key epoch, key derived in the browser — the server stores only ciphertext); the WebSocket delivery path; the clean single-page UI.

4. Load Balancer Design (sys1)

`lb/loadbalancer.py` — pure Python 3 standard library (full source in §5). It is the Assignment-5 balancer extended in four areas.

4.1 Dynamic backend selection

Every backend carries live bookkeeping: an **exponentially weighted moving average of its response time** (`ewma_ms`, $\alpha = 0.2$, fed by every proxied request *and* by the health probe's own latency, so an idle backend keeps a fresh estimate), its **in-flight request count**, and the **system load it reports** in `/health` (process CPU %, container-wide cgroup CPU % against its quota, 1-minute load average, event-loop lag). The score is

```
score(b) = EWMA_rt(b) × (1 + in_flight(b)) × (1 + load_weight × cpu_load(b)) / weight(b)
(× 2 while the backend is DEGRADED; 0 for a backend that has never been sampled)
```

and the `adaptive` algorithm (default) uses **power-of-two-choices**: sample two backends at random, keep the lower score. This is the standard remedy for the herding problem of a pure "pick the minimum" rule — with sequential traffic every backend has zero in-flight requests and a pure minimum sends everything to whichever backend happened to have the lowest EWMA; with two random choices, equal backends split evenly while a slower or more loaded backend loses every pairing it is drawn into and is left with about $1/N^2$ of the traffic until its score improves. A never-sampled backend scores 0, so a **newly added backend receives traffic immediately**; a 5 % exploration share keeps every backend measured. `least_response_time` (pure minimum, no load term) and the four Assignment-5 algorithms (`round_robin`, `least_connections`, `weighted_round_robin`, `ip_hash`) remain selectable — the algorithm comparison in §9 uses that.

4.2 Health monitoring

A background thread probes every backend's `/health` every 3 s (6 s timeout) and reads the load JSON. States instead of a boolean:

State	Meaning	Routable?
UP	answering in time	yes
DEGRADED	answering, but probe slower than 2 s or reported CPU $\geq$ 95 %	yes, score doubled
DOWN	connection refused / timeouts for 2 consecutive checks; back to UP after 2 successes	no
DRAINING	asked to leave (<code>/lb/deregister</code>): finishes in-flight, gets nothing new	no

Passive checks eject a backend immediately when a live proxy attempt cannot connect; the request is then retried on another backend (only if nothing was sent upstream yet — a POST can never be duplicated by the balancer, and even if it were, the message id makes it harmless). **Fail-open:** the last routable backend is never ejected — a slow-but-alive backend beats an empty pool. "Slow $\neq$ dead" fixes the ejection churn I diagnosed in Assignment 5 (a saturated 1-core backend answered its health check late, was ejected, and the survivors collapsed in turn).

4.3 Dynamic membership — detecting backends started while running

Three mechanisms, any one of which is sufficient:

1. **Self-registration (push).** Each backend POSTs `/lb/register {id, host, port, weight}` with a shared token at boot and every 5 s as a heartbeat. An unknown backend is admitted on the spot (`added` event). On SIGTERM the backend POSTs `/lb/deregister` and drains — graceful scale-down.
2. **Candidate scan (pull).** `lb.conf.json` lists candidate slots (sys3:3000, sys4:3001). Every 5 s the balancer probes the ones it does not know; any that answers `/health` with the expected application version is admitted (`source = scan`). This found a backend even when the backend's own registration was disabled — and, during development, it correctly *refused* the old Assignment-5 backend still running on sys3 because it reports a different version.
3. **Config watch.** The configuration file is re-read whenever its modification time changes (also `SIGHUP` and `POST /lb/reload`); `POST /lb/config` switches the algorithm or tuning at runtime.

Dynamically discovered backends that stay DOWN for 15 min are pruned; statically configured ones are kept as DOWN.

4.4 Observability

`/lb/stats` (per-backend state, source, score, EWMA, probe latency, load, in-flight, counts, percentiles, plus `active_backends`), `/lb/events` (timeline of `added` / `degraded` / `ejected` / `readmitted` / `draining` / `removed` / `config` events), the auto-refreshing dashboard at `/lb/` (screenshot §11), and a CSV access log with the active-backend count on every line.

5. Load Balancer — Full Source Code

```

1  #!/usr/bin/env python3
2  """
3  loadbalancer.py – DYNAMIC HTTP/WebSocket reverse-proxy load balancer (v2).
4
5  Assignment 6, CSD course. Runs on sys1 and spreads traffic arriving at
6  http://10.1.75.53:3269 across the chat backends on sys2/sys3/sys4.
7  Pure Python 3 standard library – no pip, no root, no external processes.
8  Extends the Assignment-5 balancer; every v1 algorithm is still available.
9
10 What is new in v2
11 -----
12 * DYNAMIC SELECTION (the assignment's core requirement). Two new algorithms:
13     adaptive (default)    score = EWMA_rt × (1 + in_flight) × (1 + cpu_load)
14                           "power of two choices": sample two backends, keep the
15                           lower score. Response time AND current system load
16                           decide, not a fixed rotation; equal backends still
17                           split evenly, a slow one is starved of new work.
18     least_response_time  score = EWMA_rt × (1 + in_flight)    (no load term)
19 EWMA_rt is fed by every proxied request and by the health probe's own
20 latency, so idle backends keep a fresh estimate. cpu_load is read from the
21 backend's /health JSON (process CPU % and 1-min load average, normalised to
22 its single core). A brand-new backend starts with score 0 (optimistic) so it
23 receives traffic immediately; a small exploration share (5 %) keeps every UP
24 backend sampled. v1 algorithms kept: round_robin, least_connections,
25 weighted_round_robin, ip_hash.
26 * HEALTH MONITORING with three states instead of a boolean:
27     UP        answering /health in time
28     DEGRADED  answering, but slowly (> degrade_ms) or reporting high load –
29               still routable, but its score is penalised (slow ≠ dead: this
30               removes the ejection churn diagnosed in Assignment 5, D-011)
31     DOWN      connection refused / timeouts for fail_threshold checks –
32               receives nothing until rise_threshold successes
33     DRAINING  asked to leave (deregister): finishes in-flight, gets nothing new
34 Passive checks: a connect failure during proxying ejects immediately; a
35 request that was not yet sent upstream is retried on another backend.
36 Fail-open: the last routable backend is never ejected.
37 * DYNAMIC MEMBERSHIP – the LB discovers backends while running:
38     - POST /lb/register {id,host,port,weight}  backends announce themselves at
39     boot and every few seconds (heartbeat); a shared token guards it
40     - POST /lb/deregister                       graceful scale-down (drain)
41     - candidate scan: `discovery.candidates` host:port slots are probed every
42     discovery_interval_s; any that answers /health is admitted
43     - lb.conf.json is re-read whenever its mtime changes (also SIGHUP, /lb/reload)
44     - dynamically added backends that stay DOWN for prune_after_s are removed
45 * OBSERVABILITY: /lb/stats (per-backend state, score, EWMA, load, counts,
46 active_backends), /lb/events (membership + health timeline), /lb/ dashboard,
47 CSV access log with the active-backend count on every line.
48
49 Usage:  python3 loadbalancer.py [path/to/lb.conf.json]
50 """
51
52 import json
53 import os
54 import random
55 import signal
56 import socket
57 import sys
58 import threading
59 import time
60 from collections import deque
61
62 # ----- configuration -----
63
64 CONF_PATH = sys.argv[1] if len(sys.argv) > 1 else os.path.join(
65     os.path.dirname(os.path.abspath(__file__)), "lb.conf.json")
66
67 DEFAULTS = {
68     "listen_host": "0.0.0.0",
69     "listen_port": 3269,
70     "algorithm": "adaptive",
71     "backends": [],          # static pool: [{"id","host","port","weight"}]
72     "health_interval_s": 3,
73     "health_timeout_s": 6,
74     "fail_threshold": 2,
75     "rise_threshold": 2,
76     "degrade_ms": 1500,    # probe slower than this -> DEGRADED

```

```

77     "degrade_cpu_pct": 90,                # backend-reported CPU above this -> DEGRADED
78     "ewma_alpha": 0.2,                    # weight of the newest sample
79     "explore_pct": 5,                     # % of adaptive picks that go to a random UP backend
80     "load_weight": 1.0,                   # multiplier on the cpu_load term of the score
81     "connect_timeout_s": 3,
82     "upstream_timeout_s": 30,
83     "register_token": "",                 # shared secret for /lb/register (empty = open)
84     "discovery": {"candidates": [], "interval_s": 5, # [{"host","port"}] slots to probe
85                  "require_version": ""},        # admit only backends reporting this /health
86     version
87     "prune_after_s": 600,                 # remove dynamic backends DOWN this long
88     "access_log": "logs/lb_access.csv",
89 }
90
91 UP, DEGRADED, DOWN, DRAINING = "UP", "DEGRADED", "DOWN", "DRAINING"
92
93
94 class Backend:
95     """One upstream server plus its live health / load / traffic bookkeeping."""
96
97     def __init__(self, cfg, source="config"):
98         self.id = cfg["id"]
99         self.host = cfg["host"]
100        self.port = int(cfg["port"])
101        self.weight = int(cfg.get("weight", 1))
102        self.source = source                # config | register | scan
103        self.state = UP                     # optimistic until the first check
104        self.consec_fail = 0
105        self.consec_ok = 0
106        self.in_flight = 0
107        self.requests = 0
108        self.errors = 0
109        self.ewma_ms = None                 # None = no sample yet -> optimistic score 0
110        self.probe_ms = None
111        self.load = {}                     # last /health "load" object from the backend
112        self.latencies = deque(maxlen=5000)
113        self.current_weight = 0             # smooth-WRR state
114        self.pool = deque()                 # idle keep-alive sockets
115        self.lock = threading.Lock()
116        self.added_at = time.time()
117        self.last_seen = time.time()        # last successful probe / heartbeat
118        self.down_since = None
119
120    def addr(self):
121        return (self.host, self.port)
122
123    def routable(self):
124        return self.state in (UP, DEGRADED)
125
126    def observe(self, ms, alpha):
127        """Feed one response-time sample into the EWMA."""
128        self.ewma_ms = ms if self.ewma_ms is None else (1 - alpha) * self.ewma_ms + alpha * ms
129
130    def cpu_load(self):
131        """0..1 : how busy the backend's SYSTEM is. Prefers the container-wide
132        cgroup CPU utilisation (sys_cpu_pct - sees other tenants of the box),
133        falls back to the process CPU % and the load average per core."""
134        cpu = float(self.load.get("cpu_pct", 0) or 0) / 100.0
135        sys_cpu = self.load.get("sys_cpu_pct")
136        if sys_cpu is not None:
137            cpu = max(cpu, float(sys_cpu) / 100.0)
138        cores = max(1, int(self.load.get("cores", 1) or 1))
139        la = float(self.load.get("loadavg1", 0) or 0) / cores
140        return max(cpu, min(la, 4.0))
141
142    def score(self, cfg):
143        """Lower is better. New backends (no sample) score 0 -> tried at once."""
144        if self.ewma_ms is None:
145            return 0.0
146        s = self.ewma_ms * (1 + self.in_flight)
147        if cfg["algorithm"] == "adaptive":
148            s *= 1 + cfg["load_weight"] * self.cpu_load()
149        if self.state == DEGRADED:
150            s *= 2.0
151        return s / max(1, self.weight)
152
153    def snapshot(self, cfg):

```

```

154     lat = sorted(self.latencies)
155     pct = lambda p: round(lat[min(len(lat) - 1, int(p / 100 * len(lat)))], 1) if lat else 0
156     return {
157         "id": self.id, "host": self.host, "port": self.port, "weight": self.weight,
158         "source": self.source, "state": self.state, "healthy": self.routable(),
159         "in_flight": self.in_flight, "requests": self.requests, "errors": self.errors,
160         "ewma_ms": round(self.ewma_ms, 1) if self.ewma_ms is not None else None,
161         "probe_ms": round(self.probe_ms, 1) if self.probe_ms is not None else None,
162         "score": round(self.score(cfg), 1),
163         "load": self.load,
164         "latency_ms": {"p50": pct(50), "p95": pct(95), "p99": pct(99)},
165         "since": round(time.time() - self.added_at),
166         "last_seen_s_ago": round(time.time() - self.last_seen, 1),
167     }
168
169
170 class LB:
171     def __init__(self):
172         self.cfg = dict(DEFAULTS)
173         self.backends = []
174         self.rr_index = 0
175         self.lock = threading.Lock()
176         self.started = time.time()
177         self.total_requests = 0
178         self.total_errors = 0
179         self.events = deque(maxlen=200)
180         self.log_lock = threading.Lock()
181         self.log_fh = None
182         self.conf_mtime = 0
183         self.reload()
184
185     # -- events -----
186     def event(self, kind, backend_id, detail=""):
187         ev = {"t": round(time.time(), 3), "kind": kind, "backend": backend_id, "detail": detail,
188             "active": len(self.routable_backends())}
189         self.events.append(ev)
190         print(f"[{b}] {kind:<12} {backend_id:<8} {detail} (active={ev['active']})", flush=True)
191
192     # -- config -----
193     def reload(self):
194         with open(CONF_PATH) as fh:
195             file_cfg = json.load(fh)
196         cfg = dict(DEFAULTS)
197         cfg.update(file_cfg)
198         disc = dict(DEFAULTS["discovery"]); disc.update(file_cfg.get("discovery", {}))
199         cfg["discovery"] = disc
200         self.conf_mtime = os.path.getmtime(CONF_PATH)
201         with self.lock:
202             old = {b.id: b for b in self.backends}
203             fresh = []
204             for bc in cfg["backends"]:
205                 if bc["id"] in old: # keep live stats across reloads
206                     b = old.pop(bc["id"])
207                     b.host, b.port = bc["host"], int(bc["port"])
208                     b.weight = int(bc.get("weight", 1))
209                     b.source = "config"
210                     fresh.append(b)
211                 else:
212                     fresh.append(Backend(bc, "config"))
213             # dynamically discovered backends survive a config reload
214             for b in old.values():
215                 if b.source != "config":
216                     fresh.append(b)
217             self.backends = fresh
218             self.cfg = cfg
219             log_path = cfg["access_log"]
220             os.makedirs(os.path.dirname(log_path) or ".", exist_ok=True)
221             new_log = not os.path.exists(log_path)
222             if self.log_fh:
223                 self.log_fh.close()
224             self.log_fh = open(log_path, "a", buffering=1)
225             if new_log:
226
227 self.log_fh.write("ts,client,method,path,backend,upstream_ms,status,bytes,active_backends\n")
228             print(f"[{b}] config loaded: algorithm={cfg['algorithm']}, "
229                 f"{len(self.backends)} backend(s): {[b.id for b in self.backends]}, "
230                 f"candidates={len(cfg['discovery']['candidates'])}", flush=True)

```

```

231
232 # -- membership -----
233 def add_backend(self, bid, host, port, weight=1, source="register"):
234     """Admit a backend announced at runtime. Returns True if it was new."""
235     with self.lock:
236         for b in self.backends:
237             if b.id == bid:
238                 b.last_seen = time.time()
239                 if (b.host, b.port) != (host, int(port)):
240                     b.host, b.port = host, int(port)
241                 if b.state == DRAINING: # came back after a drain
242                     b.state = UP; b.consec_fail = 0; b.down_since = None
243                     self.event("readmitted", bid, "re-registered after drain")
244                 return False
245             b = Backend({"id": bid, "host": host, "port": port, "weight": weight}, source)
246             self.backends.append(b)
247         self.event("added", bid, f"{host}:{port} via {source}")
248         return True
249
250 def drain_backend(self, bid):
251     with self.lock:
252         for b in self.backends:
253             if b.id == bid and b.state != DRAINING:
254                 b.state = DRAINING
255                 self.event("draining", bid, "deregister request")
256             return True
257         return False
258
259 def remove_backend(self, bid):
260     with self.lock:
261         before = len(self.backends)
262         self.backends = [b for b in self.backends if b.id != bid]
263         if len(self.backends) < before:
264             self.event("removed", bid, "")
265         return True
266     return False
267
268 # -- backend selection -----
269 def routable_backends(self):
270     return [b for b in self.backends if b.routable()]
271
272 def pick(self, client_ip, exclude=()):
273     """Choose a backend for this request using the configured algorithm."""
274     pool = [b for b in self.routable_backends() if b.id not in exclude]
275     if not pool:
276         return None
277     algo = self.cfg["algorithm"]
278     with self.lock:
279         if algo in ("adaptive", "least_response_time"):
280             fresh = [b for b in pool if b.ewma_ms is None]
281             if fresh: # never-sampled backend: try it now
282                 return fresh[0]
283             if len(pool) > 1 and random.random() * 100 < self.cfg["explore_pct"]:
284                 return random.choice(pool) # exploration keeps estimates fresh
285             if algo == "least_response_time": # pure minimum (herds under light load)
286                 return min(pool, key=lambda b: (b.score(self.cfg), b.in_flight, b.requests))
287             # adaptive = "power of two choices": sample two backends at random and
288             # keep the better score. Equal backends split evenly (no herding onto
289             # one lucky EWMA); a slow or loaded backend loses every pairing and is
290             # left with ~1/N^2 of the traffic until its score improves.
291             a, b = random.choice(pool), random.choice(pool)
292             return min((a, b), key=lambda x: (x.score(self.cfg), x.in_flight, x.requests))
293         if algo == "least_connections":
294             return min(pool, key=lambda b: (b.in_flight, b.requests))
295         if algo == "ip_hash":
296             return pool[hash(client_ip) % len(pool)]
297         if algo == "weighted_round_robin":
298             total = sum(b.weight for b in pool)
299             for b in pool:
300                 b.current_weight += b.weight
301             chosen = max(pool, key=lambda b: b.current_weight)
302             chosen.current_weight -= total
303             return chosen
304         # default: round_robin
305         self.rr_index = (self.rr_index + 1) % len(pool)
306         return pool[self.rr_index]
307

```

```

308     def pick_sticky(self, client_ip, tried):
309         """WebSocket connections use ip_hash so reconnects re-pin."""
310         pool = [b for b in self.routable_backends() if b.id not in tried]
311         if not pool:
312             return None
313         return pool[hash(client_ip) % len(pool)]
314
315     # -- access log -----
316     def log(self, client, method, path, backend_id, ms, status, nbytes):
317         with self.log_lock:
318             self.log_fh.write(f"{time.time():.3f},{client},{method},{path},{backend_id},"
319                               f"{ms:.1f},{status},{nbytes},{len(self.routable_backends())}\n")
320
321
322 LB_STATE = LB()
323
324 # ----- health + discovery -----
325
326 def probe(host, port, timeout):
327     """GET /health. Returns (ok, ms, json_or_None)."""
328     t0 = time.time()
329     try:
330         with socket.create_connection((host, port), timeout=timeout) as s:
331             s.sendall(b"GET /health HTTP/1.1\r\nHost: lb-health\r\nConnection: close\r\n\r\n")
332             s.settimeout(timeout)
333             buf = b""
334             while True:
335                 chunk = s.recv(65536)
336                 if not chunk:
337                     break
338                 buf += chunk
339                 if len(buf) > 65536:
340                     break
341             ms = (time.time() - t0) * 1000
342             head, _, body = buf.partition(b"\r\n\r\n")
343             ok = head.startswith(b"HTTP/1.1 200") or head.startswith(b"HTTP/1.0 200")
344             data = None
345             try:
346                 data = json.loads(body.decode() or "null")
347             except ValueError:
348                 pass
349             return ok, ms, data
350     except OSError:
351         return False, (time.time() - t0) * 1000, None
352
353
354 def health_loop():
355     """Active checks: probe every backend, keep EWMA/load fresh, move states."""
356     while True:
357         cfg = LB_STATE.cfg
358         for b in list(LB_STATE.backends()):
359             ok, ms, data = probe(b.host, b.port, cfg["health_timeout_s"])
360             if ok:
361                 b.probe_ms = ms
362                 b.last_seen = time.time()
363                 if isinstance(data, dict) and isinstance(data.get("load"), dict):
364                     b.load = data["load"]
365                 # an idle backend's only latency signal is the probe itself
366                 if b.in_flight == 0:
367                     b.observe(ms, cfg["ewma_alpha"])
368                 b.consec_ok += 1
369                 b.consec_fail = 0
370                 slow = ms > cfg["degrade_ms"] or b.cpu_load() * 100 >= cfg["degrade_cpu_pct"]
371                 if b.state == DOWN and b.consec_ok >= cfg["rise_threshold"]:
372                     b.state = DEGRADED if slow else UP
373                     b.down_since = None
374                     LB_STATE.event("readmitted", b.id, f"probe {ms:.0f} ms")
375                 elif b.state == UP and slow:
376                     b.state = DEGRADED
377                     LB_STATE.event("degraded", b.id, f"probe {ms:.0f} ms, cpu
378 {b.cpu_load()*100:.0f}%")
379                 elif b.state == DEGRADED and not slow:
380                     b.state = UP
381                     LB_STATE.event("recovered", b.id, f"probe {ms:.0f} ms")
382             else:
383                 b.consec_fail += 1
384                 b.consec_ok = 0

```

```

385         if b.routable() and b.consec_fail >= cfg["fail_threshold"]:
386             others = [x for x in LB_STATE.backends if x is not b and x.routable()]
387             if others:
388                 b.state = DOWN
389                 b.down_since = time.time()
390                 LB_STATE.event("ejected", b.id, f"{b.consec_fail} failed checks")
391             else:
392                 print(f"[lb] {b.id} failing checks but is the last one – keeping it (fail-
393 open)", flush=True)
394             elif b.state == DRAINING and b.consec_fail >= cfg["fail_threshold"]:
395                 b.state = DOWN
396                 b.down_since = time.time()
397                 LB_STATE.event("ejected", b.id, "drained backend went away")
398                 # prune dynamic members that have been dead for a long time
399                 if b.state == DOWN and b.source != "config" and b.down_since and \
400                     time.time() - b.down_since > cfg["prune_after_s"]:
401                     LB_STATE.remove_backend(b.id)
402                 time.sleep(cfg["health_interval_s"])
403
404
405 def discovery_loop():
406     """Pull-side discovery: probe candidate slots; admit any that answer."""
407     while True:
408         cfg = LB_STATE.cfg
409         known = {(b.host, b.port) for b in LB_STATE.backends}
410         for c in cfg["discovery"].get("candidates", []):
411             host, port = c["host"], int(c["port"])
412             if (host, port) in known:
413                 continue
414             ok, ms, data = probe(host, port, min(2.0, cfg["health_timeout_s"]))
415             if ok:
416                 want = cfg["discovery"].get("require_version")
417                 if want and not str((data or {}).get("version", "")).startswith(want):
418                     # some other service on that slot – not ours
419                     continue
420                 bid = (data or {}).get("backend") or c.get("id") or f"{host}:{port}"
421                 LB_STATE.add_backend(bid, host, port, c.get("weight", 1), source="scan")
422             # config file edited? reload without a restart
423             try:
424                 if os.path.getmtime(CONF_PATH) != LB_STATE.conf_mtime:
425                     LB_STATE.reload()
426             except OSError:
427                 pass
428             time.sleep(cfg["discovery"].get("interval_s", 5))
429
430 def eject_now(b):
431     """Passive check: a live proxy attempt failed – eject immediately (fail-open)."""
432     if b.routable():
433         others = [x for x in LB_STATE.backends if x is not b and x.routable()]
434         if not others:
435             return
436         b.state = DOWN
437         b.down_since = time.time()
438         b.consec_fail = LB_STATE.cfg["fail_threshold"]
439         b.consec_ok = 0
440         LB_STATE.event("ejected", b.id, "passive: connect/proxy failure")
441
442
443 # ----- HTTP plumbing -----
444
445 HOP_BY_HOP = {"connection", "keep-alive", "proxy-authenticate", "proxy-authorization",
446               "te", "trailers", "transfer-encoding", "upgrade"}
447
448
449 def read_head(sock_file):
450     first = sock_file.readline(65536)
451     if not first or first in (b"\r\n", b"\n"):
452         return None, None, None
453     headers, hmap = [], {}
454     while True:
455         line = sock_file.readline(65536)
456         if line in (b"\r\n", b"\n", b""):
457             break
458         headers.append(line)
459         k, _, v = line.partition(b":")
460         hmap[k.strip().lower()] = v.strip()
461     return first.rstrip(b"\r\n"), headers, hmap

```

```

462
463
464 def get_upstream(backend, timeout):
465     """Reuse a pooled keep-alive socket if one is idle, else connect fresh."""
466     with backend.lock:
467         while backend.pool:
468             s = backend.pool.popleft()
469             s.setblocking(False)
470             try:
471                 if s.recv(1, socket.MSG_PEEK):
472                     s.close(); continue
473                 s.close(); continue
474             except BlockingIOError:
475                 s.setblocking(True); s.settimeout(timeout)
476                 return s, True
477             except OSError:
478                 try: s.close()
479                 except OSError: pass
480             s = socket.create_connection(backend.addr(), timeout=LB_STATE.cfg["connect_timeout_s"])
481             s.settimeout(timeout)
482             return s, False
483
484
485 def pump(src, dst):
486     """Copy bytes one way until EOF/error (WebSocket tunnelling)."""
487     try:
488         while True:
489             data = src.recv(65536)
490             if not data:
491                 break
492             dst.sendall(data)
493     except OSError:
494         pass
495     finally:
496         for s in (src, dst):
497             try: s.shutdown(socket.SHUT_RDWR)
498             except OSError: pass
499             try: s.close()
500             except OSError: pass
501
502
503 def build_upstream_head(first, headers, hmap, client_ip, is_ws):
504     out = [first + b"\r\n"]
505     for line in headers:
506         k = line.split(b":", 1)[0].strip().lower()
507         if k in HOP_BY_HOP or k in (b"x-forwarded-for", b"x-real-ip", b"x-forwarded-proto"):
508             continue
509         out.append(line)
510         prior = hmap.get(b"x-forwarded-for")
511         out.append(b"X-Forwarded-For: " + ((prior + b", ") if prior else b"") + client_ip.encode() +
512 b"\r\n")
513         out.append(b"X-Real-IP: " + client_ip.encode() + b"\r\n")
514         out.append(b"X-Forwarded-Proto: http\r\n")
515         out.append(b"Connection: Upgrade\r\nUpgrade: websocket\r\n" if is_ws else b"Connection: keep-
516 alive\r\n")
517         out.append(b"\r\n")
518         return b"".join(out)
519
520
521 # ----- request handling -----
522
523 def handle_client(client, client_addr):
524     client_ip = client_addr[0]
525     client.settimeout(60)
526     cf = client.makefile("rb")
527     try:
528         while True:
529             first, headers, hmap = read_head(cf)
530             if first is None:
531                 return
532             try:
533                 method, path, _version = first.split(b" ", 2)
534             except ValueError:
535                 return
536             LB_STATE.total_requests += 1
537
538             body = b""

```

```

539         clen = int(hmap.get(b"content-length", b"0") or 0)
540     if clen:
541         body = cf.read(clen)
542
543     if path.startswith(b"/lb/") or path == b"/lb/":
544         if not serve_admin(client, method, path, hmap, body, client_ip):
545             return
546         continue
547
548     is_ws = (b"upgrade" in hmap.get(b"connection", b"").lower()
549             and hmap.get(b"upgrade", b"").lower() == b"websocket")
550
551     tried = set()
552     upstream = pooled = backend = None
553     while True:
554         backend = LB_STATE.pick(client_ip, exclude=tried) if not is_ws \
555             else LB_STATE.pick_sticky(client_ip, tried)
556         if backend is None:
557             send_error(client, 503, "No healthy backends")
558             LB_STATE.total_errors += 1
559             LB_STATE.log(client_ip, method.decode(), path.decode(), "-", 0.0, 503, 0)
560             return
561         try:
562             upstream, pooled = get_upstream(backend, LB_STATE.cfg["upstream_timeout_s"])
563             break
564         except OSError:
565             tried.add(backend.id)
566             eject_now(backend) # passive ejection + safe retry (nothing sent yet)
567             continue
568
569     head = build_upstream_head(first, headers, hmap, client_ip, is_ws)
570     backend.in_flight += 1
571     backend.requests += 1
572     t0 = time.time()
573     try:
574         upstream.sendall(head + body)
575         if is_ws:
576             threading.Thread(target=pump, args=(client, upstream), daemon=True).start()
577             pump(upstream, client)
578             LB_STATE.log(client_ip, "WS", path.decode(), backend.id, (time.time() - t0) *
579 1000, 101, 0)
580         return
581
582     uf = upstream.makefile("rb")
583     rfirst, rheaders, rhmap = read_head(uf)
584     if rfirst is None:
585         raise OSError("upstream sent no response")
586     status = int(rfirst.split(b" ")[1])
587     r_clen = rhmap.get(b"content-length")
588     keep_up = rhmap.get(b"connection", b"keep-alive").lower() != b"close" and r_clen is
589 not None
590
591     out = [rfirst + b"\r\n"]
592     for line in rheaders:
593         if line.split(b":", 1)[0].strip().lower() == b"connection":
594             continue
595         out.append(line)
596     out.append(b"X-LB-Active-Backends: " +
597 str(len(LB_STATE.routable_backends())).encode() + b"\r\n")
598     out.append(b"Connection: keep-alive\r\n" if r_clen is not None else b"Connection:
599 close\r\n")
600     out.append(b"\r\n")
601     client.sendall(b"".join(out))
602
603     nbytes = 0
604     if r_clen is not None:
605         remaining = int(r_clen)
606         while remaining:
607             chunk = uf.read(min(65536, remaining))
608             if not chunk:
609                 raise OSError("upstream truncated body")
610             client.sendall(chunk); nbytes += len(chunk); remaining -= len(chunk)
611     else:
612         while True:
613             chunk = uf.read1(65536)
614             if not chunk:
615                 break

```

```

616         client.sendall(chunk); nbytes += len(chunk)
617
618     ms = (time.time() - t0) * 1000
619     backend.latencies.append(ms)
620     backend.observe(ms, LB_STATE.cfg["ewma_alpha"])    # the dynamic signal
621     if status >= 500:
622         backend.errors += 1
623     LB_STATE.log(client_ip, method.decode(), path.decode(), backend.id, ms, status,
624 nbytes)
625
626     if keep_up:
627         with backend.lock:
628             if len(backend.pool) < 32:
629                 upstream.settimeout(LB_STATE.cfg["upstream_timeout_s"])
630                 backend.pool.append(upstream)
631             else:
632                 upstream.close()
633         else:
634             upstream.close()
635         if r_clen is None:
636             return
637     except OSError:
638         backend.errors += 1
639         LB_STATE.total_errors += 1
640         eject_now(backend)
641         try: upstream.close()
642         except OSError: pass
643     LB_STATE.log(client_ip, method.decode(), path.decode(), backend.id, (time.time() -
644 t0) * 1000, 502, 0)
645     send_error(client, 502, "Upstream failed mid-request")
646     return
647     finally:
648         backend.in_flight -= 1
649     except OSError:
650         pass
651     finally:
652         try:
653             cf.close(); client.close()
654         except OSError:
655             pass
656
657
658 # ----- admin endpoints -----
659
660 def stats_dict():
661     lb = LB_STATE
662     return {
663         "algorithm": lb.cfg["algorithm"], "version": "v2-dynamic",
664         "uptime_s": round(time.time() - lb.started, 1),
665         "total_requests": lb.total_requests, "total_errors": lb.total_errors,
666         "active_backends": len(lb.routable_backends()),
667         "backends": [b.snapshot(lb.cfg) for b in lb.backends],
668         "candidates": lb.cfg["discovery"].get("candidates", []),
669         "recent_events": list(lb.events)[-10:],
670     }
671
672
673 DASHBOARD = """<!doctype html><meta charset=utf-8>
674 <meta http-equiv=refresh content=2>
675 <title>Dynamic LB dashboard</title>
676 <style>
677   body{font:14px -apple-system, Segoe UI, Roboto, sans-serif;margin:30px auto;max-
678 width:900px;color:#1a1a1e}
679   h1{font-size:20px;margin-bottom:4px} table{border-collapse:collapse;width:100%;margin-top:10px}
680   td,th{border:1px solid #ddd;padding:6px 8px;text-align:left;font-variant-numeric:tabular-
681 nums;font-size:13px}
682   th{background:#f4f4f6}.UP{color:#2e7d32;font-weight:600}.DEGRADED{color:#b26a00;font-weight:600}
683   .DOWN{color:#c62828;font-weight:600}.DRAINING{color:#7a4fd0;font-weight:600}.muted{color:#777}
684   .bar{background:#e8ebff;height:8px;border-radius:4px;overflow:hidden}.bar
685   i{display:block;height:8px;background:#4f6df5}
686   ul{font-family:ui-monospace, monospace;font-size:12px;padding-left:18px}
687 </style>
688 <h1>Dynamic load balancer - sys1:3269</h1>
689 <p class=muted>algorithm <b>%(algo)s</b> · <b>%(active)d</b> active backend(s)</b> of <b>%(n)d</b> · uptime %
690 (up).0fs · <b>%(tot)d</b> requests · <b>%(err)d</b> errors · auto-refresh 2s</p>
691 <table><tr><th>backend</th><th>addr</th><th>source</th><th>state</th><th>score ↓</th><th>EWMA
692 ms</th><th>probe ms</th>

```

```

693 <th>cpu %</th><th>load1</th><th>in-flight</th><th>requests</th><th>errors</th><th>p50/p95 ms</th>
694 <th>share</th></tr>%<(rows)s</table>
695 <h3>Recent events</h3><ul>%<(events)s</ul>
696 <p class=muted>JSON: <a href=/lb/stats>/lb/stats</a> · <a href=/lb/events>/lb/events</a> · reload
697 config: POST /lb/reload · register: POST /lb/register</p>"""
698
699
700 def serve_admin(client, method, path, hmap, body, client_ip):
701     """Handle /lb/* requests. Returns False to close the connection."""
702     code = 200
703     lb = LB_STATE
704     if path == b"/lb/stats":
705         out, ctype = json.dumps(stats_dict(), indent=2).encode(), b"application/json"
706     elif path == b"/lb/events":
707         out, ctype = json.dumps({"events": list(lb.events)}, indent=2).encode(),
708 b"application/json"
709     elif path in (b"/lb", b"/lb/"):
710         total = sum(b.requests for b in lb.backends) or 1
711         rows = ""
712         for b in lb.backends:
713             s = b.snapshot(lb.cfg)
714             share = 100.0 * b.requests / total
715             rows += (<tr><td>%s</td><td>%s</td><td>%s</td><td class=%s>%s</td><td>%s</td>
716 <td>%s</td><td>%s</td><td>%d</td><td>%d</td><td>%d</td><td>%d</td><td>%d</td><td>%0f / %0f</td>
717 <td><div class=bar><i style='width:%0f%%'></i></div>%0f%</td></tr>" % (
718             s["id"], s["host"], s["port"], s["source"], s["state"], s["state"],
719             s["score"], s["ewma_ms"] if s["ewma_ms"] is not None else "-",
720             s["probe_ms"] if s["probe_ms"] is not None else "-",
721             s["load"].get("cpu_pct", "-"), s["load"].get("loadavg1", "-"),
722             s["in_flight"], s["requests"], s["errors"],
723             s["latency_ms"]["p50"], s["latency_ms"]["p95"], share, share))
724         events = "".join("<li>%s %011s %07s %s (active=%d)</li>" % (
725             time.strftime("%H:%M:%S", time.localtime(e["t"])), e["kind"], e["backend"],
726 e["detail"], e["active"]))
727         for e in list(lb.events)[-12:][::-1])
728         out = (DASHBOARD % {"algo": lb.cfg["algorithm"], "active": len(lb.routable_backends()),
729             "n": len(lb.backends), "up": time.time() - lb.started,
730             "tot": lb.total_requests, "err": lb.total_errors,
731             "rows": rows, "events": events}).encode()
732         ctype = b"text/html; charset=utf-8"
733     elif path in (b"/lb/register", b"/lb/deregister") and method == b"POST":
734         token = lb.cfg.get("register_token", "")
735         if token and hmap.get(b"x-lb-token", b"").decode() != token:
736             out, ctype, code = b'{"ok":false,"error":"bad token"}', b"application/json", 403
737         else:
738             try:
739                 req = json.loads(body.decode() or "{}")
740                 bid = str(req["id"])
741                 if path == b"/lb/register":
742                     host = req.get("host") or client_ip
743                     added = lb.add_backend(bid, host, int(req["port"]), int(req.get("weight", 1)))
744                     out = json.dumps({"ok": True, "added": added, "active_backends":
745 len(lb.routable_backends())}).encode()
746                 else:
747                     out = json.dumps({"ok": True, "draining": lb.drain_backend(bid)}).encode()
748                     ctype = b"application/json"
749                 except (KeyError, ValueError, TypeError) as e:
750                     out, ctype, code = json.dumps({"ok": False, "error": f"bad request:
751 {e}"}).encode(), b"application/json", 400
752                 elif path.startswith(b"/lb/backends/") and method == b"DELETE":
753                     bid = path[len(b"/lb/backends/"):].decode()
754                     out, ctype = json.dumps({"ok": lb.remove_backend(bid)}).encode(), b"application/json"
755                 elif path == b"/lb/config" and method == b"POST":
756                     # runtime tuning, e.g. {"algorithm":"round_robin"} - merged, not persisted
757                     try:
758                         patch = json.loads(body.decode() or "{}")
759                         allowed = {"algorithm", "explore_pct", "load_weight", "degrade_ms", "ewma_alpha",
760 "health_interval_s"}
761                         applied = {k: v for k, v in patch.items() if k in allowed}
762                         lb.cfg.update(applied)
763                         lb.event("config", "-", json.dumps(applied))
764                         out, ctype = json.dumps({"ok": True, "applied": applied}).encode(), b"application/json"
765                     except ValueError as e:
766                         out, ctype, code = json.dumps({"ok": False, "error": str(e)}).encode(),
767 b"application/json", 400
768                 elif path == b"/lb/config":

```

```

770         out, ctype = json.dumps({k: v for k, v in lb.cfg.items() if k != "register_token"},
771 indent=2).encode(), b"application/json"
772         elif path == b"/lb/reload" and method == b"POST":
773             try:
774                 lb.reload()
775                 out, ctype = b'{"ok":true}', b"application/json"
776             except Exception as e:
777                 out, ctype = json.dumps({"ok": False, "error": str(e)}).encode(), b"application/json"
778         elif path == b"/lb/health":
779             out, ctype = json.dumps({"status": "ok", "service": "lb", "version": "v2-dynamic",
780 "active_backends": len(lb.routable_backends())},
781 separators=(",", ":")).encode(), b"application/json"
782         else:
783             out, ctype, code = b'{"error":"no such admin route"}', b"application/json", 404
784             client.sendall(b"HTTP/1.1 " + str(code).encode() + b" OK\r\nContent-Type: " + ctype +
785 b"\r\nContent-Length: " + str(len(out)).encode() +
786 b"\r\nCache-Control: no-store\r\nConnection: keep-alive\r\n\r\n" + out)
787         return True
788
789
790 def send_error(client, code, msg):
791     out = json.dumps({"error": msg}).encode()
792     try:
793         client.sendall(b"HTTP/1.1 " + str(code).encode() + b" LB Error\r\nContent-Type:
794 application/json\r\n"
795 b"Content-Length: " + str(len(out)).encode() + b"\r\nConnection:
796 close\r\n\r\n" + out)
797     except OSError:
798         pass
799
800
801 # ----- main -----
802
803 def main():
804     signal.signal(signal.SIGHUP, lambda *_: LB_STATE.reload())
805     threading.Thread(target=health_loop, daemon=True).start()
806     threading.Thread(target=discovery_loop, daemon=True).start()
807
808     srv = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
809     srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
810     srv.bind((LB_STATE.cfg["listen_host"], LB_STATE.cfg["listen_port"]))
811     srv.listen(512)
812     print(f"[lb] v2 listening on {LB_STATE.cfg['listen_host']}:{LB_STATE.cfg['listen_port']} "
813 f"-- dashboard at /lb/", flush=True)
814     while True:
815         try:
816             client, addr = srv.accept()
817             except OSError as e:
818                 print(f"[lb] accept error (surviving): {e}", flush=True)
819                 time.sleep(0.2)
820                 continue
821             client.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)
822             threading.Thread(target=handle_client, args=(client, addr), daemon=True).start()
823
824 if __name__ == "__main__":
825     main()

```

6. Database Persistence and Duplicate Prevention

6.1 Persistent shared storage

All backends are stateless; every user, session, room and message lives in **one SQLite database (WAL mode)** owned by `app/db_service.js` on `sys1` and reached over HTTP by every backend. The database file (`~/assignment6/data/chat.sqlite`) survives restarts of any backend, of the database service, and of the whole cluster — verified in the smoke test (48 assertions, including a database restart mid-test) and live (\$9.5). Assignment 5's data (717 users, 5 rooms, 111 180 messages) was migrated in on first boot, so the old accounts still work. SQLite was chosen because the lab boxes allow no root and no package installation; Node

22's built-in `node:sqlite` needs nothing but a user-local Node binary, and a real relational engine gives real constraints.

6.2 Unique message ids

Every message has a **UUID v4 id minted by the client when the message is composed** and reused on every retry (body field `id` or `Idempotency-Key` header). Only the client knows that two sends are "the same message" — a server-minted id cannot recognise a retry — so this is the right place for the id. A client that sends none gets one minted by the backend (still unique; but then a retry is, by definition, a new message).

6.3 How duplicates are prevented — three layers

Layer	Where	Mechanism
1 · fast path	each backend	LRU of the last 5 000 stored ids (also fed by the firehose, so a retry that lands on a <i>different</i> backend is usually caught here too) → <code>200 {duplicate:true}</code> without touching the database
2 · authoritative	database	<code>messages.id</code> TEXT PRIMARY KEY ; <code>POST /messages</code> runs <code>SELECT ... ; INSERT ... ON CONFLICT(id) DO NOTHING</code> inside one <code>BEGIN IMMEDIATE</code> transaction. Two backends sending the same id at the same instant are serialised; the loser gets the stored row back with <code>duplicate:true</code> and the rejection is written to <code>dedup_log</code>
3 · presentation	browser	renders each id once; a retried send that the server deduplicated never appears twice

A duplicate is answered with **HTTP 200, `duplicate:true`, the original seq and id, and header `x-Duplicate: 1`** — an idempotent success, never an error the client would retry again. Per-room ordering uses `UNIQUE(room, seq)` so sequences are gap-free. Schema excerpt and the append transaction:

```

1  function log(...a) { if (process.env.LOG_LEVEL !== 'silent') console.log('[db]', ...a); }
2
3  // — Schema —————
4  const db = new DatabaseSync(DB_FILE);
5  db.exec(`
6      PRAGMA journal_mode = WAL;
7      PRAGMA synchronous = NORMAL;
8      PRAGMA foreign_keys = ON;
9      CREATE TABLE IF NOT EXISTS users (username TEXT PRIMARY KEY, hash TEXT NOT NULL, created
10     INTEGER NOT NULL);
11     CREATE TABLE IF NOT EXISTS sessions (token TEXT PRIMARY KEY, username TEXT NOT NULL, created
12     INTEGER NOT NULL, expires INTEGER NOT NULL);
13     CREATE TABLE IF NOT EXISTS rooms (id TEXT PRIMARY KEY, locked INTEGER NOT NULL DEFAULT 0,
14     epoch INTEGER NOT NULL DEFAULT 0,
15                                     verifier TEXT, created INTEGER NOT NULL, created_by TEXT);
16     CREATE TABLE IF NOT EXISTS messages (
17         id TEXT PRIMARY KEY,          -- globally unique message id (UUID v4): THE duplicate
18     guard
19         room TEXT NOT NULL,
20         seq INTEGER NOT NULL,        -- per-room sequence number, gap-free
21         sender TEXT NOT NULL,
22         ts INTEGER NOT NULL,         -- client/backend timestamp (ms)
23         kind TEXT NOT NULL,         -- plain | enc | system
24         text TEXT,                  -- plaintext rooms only
25         env TEXT,                   -- E2E envelope JSON (ciphertext) for locked rooms
26         via TEXT,                   -- backend that accepted it
27         stored_at INTEGER NOT NULL,  -- server receive time (ms)
28         extra TEXT,                  -- optional JSON (e.g. roomMeta on system messages)
29         UNIQUE(room, seq)
30     );
31     CREATE INDEX IF NOT EXISTS idx_messages_room_seq ON messages(room, seq);
32     CREATE TABLE IF NOT EXISTS dedup_log (          -- audit trail of rejected duplicates
33         n INTEGER PRIMARY KEY AUTOINCREMENT, id TEXT NOT NULL, room TEXT, via TEXT, at INTEGER NOT
34     NULL);
35     CREATE INDEX IF NOT EXISTS idx_sessions_expires ON sessions(expires);
36 `);
37
38  const q = {
39     getUser: db.prepare('SELECT username, hash, created FROM users WHERE username = ?'),
40     putUser: db.prepare('INSERT INTO users(username, hash, created) VALUES(?, ?, ?) ON
41     CONFLICT(username) DO UPDATE SET hash = excluded.hash'),
42     delUser: db.prepare('DELETE FROM users WHERE username = ?'),
43     getSess: db.prepare('SELECT token, username, created, expires FROM sessions WHERE token = ?'),
44     putSess: db.prepare('INSERT INTO sessions(token, username, created, expires) VALUES(?, ?, ?, ?)
45     ON CONFLICT(token) DO UPDATE SET expires = excluded.expires'),
46     delSess: db.prepare('DELETE FROM sessions WHERE token = ?'),
47     getRoom: db.prepare('SELECT id, locked, epoch, verifier, created, created_by FROM rooms WHERE id
48     = ?'),
49     putRoom: db.prepare('INSERT INTO rooms(id, locked, epoch, verifier, created, created_by)
50     VALUES(?, ?, ?, ?, ?, ?)
51     ON CONFLICT(id) DO UPDATE SET locked = excluded.locked, epoch =
52     excluded.epoch, verifier = excluded.verifier'),
53     delRoom: db.prepare('DELETE FROM rooms WHERE id = ?'),
54     allRooms: db.prepare('SELECT id, locked, epoch, verifier, created, created_by FROM rooms ORDER BY
55     id'),
56     nextSeq: db.prepare('SELECT COALESCE(MAX(seq), 0) + 1 AS seq FROM messages WHERE room = ?'),
57     insMsg: db.prepare('INSERT INTO messages(id, room, seq, sender, ts, kind, text, env, via,
58     stored_at, extra)
59     VALUES(?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?) ON CONFLICT(id) DO NOTHING'),
60     getMsg: db.prepare('SELECT * FROM messages WHERE id = ?'),
61     listMsgs: db.prepare('SELECT * FROM messages WHERE room = ? AND seq > ? ORDER BY seq DESC LIMIT
62     ?'),
63     countRoom: db.prepare('SELECT COUNT(*) AS n, COALESCE(MAX(seq), 0) AS last FROM messages WHERE
64     room = ?'),
65     countAll: db.prepare('SELECT COUNT(*) AS n FROM messages'),
66     countUsers: db.prepare('SELECT COUNT(*) AS n FROM users'),
67     countRooms: db.prepare('SELECT COUNT(*) AS n FROM rooms'),
68     countDups: db.prepare('SELECT COUNT(*) AS n FROM dedup_log'),
69     logDup: db.prepare('INSERT INTO dedup_log(id, room, via, at) VALUES(?, ?, ?, ?)'),
70     recentDups: db.prepare('SELECT id, room, via, at FROM dedup_log ORDER BY n DESC LIMIT ?'),
71     purgeSess: db.prepare('DELETE FROM sessions WHERE expires < ?'),
72 };
73
74  const stats = { requests: 0, appended: 0, duplicates_rejected: 0, started };
75
76  function rowToEntry(r) {

```

```

77   const e = { id: r.id, seq: r.seq, room: r.room, from: r.sender, ts: r.ts, kind: r.kind, via:
78   r.via };
79   if (r.kind === 'enc') e.env = JSON.parse(r.env);
80   else e.text = r.text;
81   if (r.extra) Object.assign(e, JSON.parse(r.extra));
82   return e;
83 }
84 function roomToRec(r) {
85   return r ? { id: r.id, locked: !!r.locked, epoch: r.epoch, verifier: r.verifier, created:
86   r.created, by: r.created_by } : null;
87 }
88
89 // — Idempotent append: the heart of the "no duplicates" guarantee —————
90 // Runs as ONE transaction. Node is single-threaded and DatabaseSync is
91 // synchronous, so two backends POSTing the same id at the same instant are
92 // serialised here; the second one finds the row and is reported as duplicate.
93 const appendTx = (room, entry) => {
94   db.exec('BEGIN IMMEDIATE');
95   try {
96     const existing = q.getMsg.get(entry.id);
97     if (existing) {
98       q.logDup.run(entry.id, room, entry.via || null, Date.now());
99       db.exec('COMMIT');
100      return { duplicate: true, rec: rowToEntry(existing) };
101    }
102    const seq = q.nextSeq.get(room).seq;
103    const extra = {};
104    for (const k of Object.keys(entry)) {
105      if (!['id', 'seq', 'room', 'from', 'ts', 'kind', 'text', 'env', 'via'].includes(k)) extra[k]
106      = entry[k];
107    }
108    const res = q.insMsg.run(entry.id, room, seq, entry.from, entry.ts || Date.now(), entry.kind,
109    entry.kind === 'enc' ? null : (entry.text ?? null),
110    entry.kind === 'enc' ? JSON.stringify(entry.env) : null,
111    entry.via || null, Date.now(), Object.keys(extra).length ? JSON.stringify(extra) : null);
112    if (res.changes === 0) { // lost a race inside the same tx window
113      const row = q.getMsg.get(entry.id);
114      q.logDup.run(entry.id, room, entry.via || null, Date.now());
115      db.exec('COMMIT');
116      return { duplicate: true, rec: rowToEntry(row) };
117    }
118    db.exec('COMMIT');
119    return { duplicate: false, rec: rowToEntry(q.getMsg.get(entry.id)) };
120  } catch (e) {
121    db.exec('ROLLBACK');
122    throw e;
123  }
124 };
125
126 // — One-time migration from the Lab 5 store (users, rooms, messages) —————
127 function migrateFromLab5(dir) {
128   const marker = path.join(DATA_DIR, '.migrated-from-lab5');
129   if (fs.existsSync(marker)) return;
130   const kvFile = path.join(dir, 'kv.json'), msgFile = path.join(dir, 'messages.jsonl');
131   if (!fs.existsSync(kvFile) && !fs.existsSync(msgFile)) { log('migration: nothing found in', dir);
132   return; }
133   let users = 0, rooms = 0, msgs = 0, dups = 0;
134   db.exec('BEGIN');

```

6.4 Verification through the public URL

scripts/dedup_test.py verifies against the **database row count**, not against what a backend said:

Test	Scenario	Sent	Stored in DB	duplicate:true responses	Backends involved	Result
T1	same id, sequential retries	20	1	19	sys2, sys3, sys4	PASS
T2	same id, concurrent storm across backends	20	1	19	sys2, sys3, sys4	PASS
T3	send, drop connection, reconnect, re-send	2	1	1	sys2, sys4	PASS
T4	Idempotency-Key header	5	1	4	sys2	PASS
T5	control: distinct ids	20	20	0	sys3	PASS

Database after the test: 112484 messages, 4 duplicates rejected in total (sqlite (node:sqlite, WAL)).

The 20-way concurrent storm is the important one: twenty connections, spread by the balancer over all three backends, POST the same id at the same instant; exactly one row exists afterwards and all twenty responses carry the same seq. The smoke test repeats this locally and additionally proves that a duplicate of an id stored *before* a database restart is still rejected (the guard is durable, not in memory), and that the browser's "Resend last" button — which re-POSTs the previous message with the same id — is answered with `duplicate:true` (screenshot §11).

7. Load Generator and Experimental Method

`loadgen/loadgen.py` (Python 3 stdlib, on my Mac) drives the public URL with a realistic mix per virtual user — 10 % login (script, the expensive one), 55 % fetch history, 30 % send, 5 % health. Every send carries a client-minted id and is retried with the same id on a transport error, exactly like the browser. Modes: **closed loop** (N users in a request→response loop), **open loop** (Poisson arrivals at a fixed *offered* rate, independent of how slow responses get), and **ramp** (a schedule of concurrency steps). Per request it records latency, status, error class, `X-Backend-Id`, `X-LB-Active-Backends` and the duplicate flag; with `--poll-stats` it samples `/lb/stats` once a second, giving the number of active backends, per-backend state, score and CPU as a time series. The first 5 s of every run are discarded; percentiles are computed over the full sample.

Experiment	What varies	Held constant
L response time vs load	concurrency 1, 10, 25, 50, 100, 200 × backends 1, 2, 3 (2 reps, 40 s)	adaptive algorithm; configurations interleaved in shuffled order inside every (level, rep) round so 1/2/3-backend comparisons share the same host conditions
O throughput vs offered load	offered 20, 50, 100, 200, 300 req/s (Poisson) × backends 1, 2, 3 (30 s)	as above
SCALE dynamic scaling	one 300 s run: 25 → 50 → 100 → 200 users; sys3 started at ~128 s and sys4 at ~184 s <i>while the run was going</i>	adaptive
FAIL failure / recovery	150 s at 100 users on 3 backends; sys3 SIGKILLED at ~30 s, restarted at ~90 s	adaptive
ALGO dynamic vs static	adaptive vs round_robin vs least_connections at 50 users while a CPU-burning process runs on sys3 (2 reps)	3 backends

The number of backends is always changed the *dynamic* way — by starting or stopping the backend process on the system (`scripts/scale.sh sysN up|down|kill`) — never by editing the balancer's configuration; the balancer must notice by itself.

8. Results — Response Time vs Load and Throughput vs Offered Load

8.1 Response time vs load (closed loop)

Concurrency	Backends	Total	Success	Failed	Err %	Throughput (req/s)	Mean ms	p50	p90	p95	p99	Active	Distribution	Speed-up vs 1

(chart `response_time_vs_load.png` pending)

(chart `throughput_vs_load.png` pending)

{{A_RT}}

8.2 Throughput vs offered load (open loop)

Offered load (req/s)	Backends	Achieved (req/s)	Success	Failed / dropped	Err %	p50 ms	p95 ms	p99 ms	Active	Distribution

(chart throughput_vs_offered_load.png pending)

{{A_OFFERED}}

9. Results — Dynamic Scaling, Failure Recovery, Algorithm Comparison

9.1 Dynamic scaling: performance after each backend is added

The system started with **one** backend (sys2). Load was raised in steps (25 → 50 → 100 users) until the single backend was saturated; then `bash scripts/scale.sh sys3 up` and later `... sys4 up` were executed on the other systems *while the load generator kept running*. The balancer admitted each new backend by itself (registration heartbeat and candidate scan both fired) and the load generator's per-second samples of `/lb/stats` show exactly when.

Phase	Window (s)	Users	Active backends	Throughput (req/s)	p50 ms	p95 ms	Errors	Share sys2 / sys3 / sys4
A: 1 backend, 25 users	5–40	25	1	95.5	75	2089	0	100% / 0% / 0%
B: 1 backend, 50 users	45–80	50	1	105.2	75	4482	0	100% / 0% / 0%
C: 1 backend, 100 users (saturated)	85–128	100	1	98.6	61	9028	0	100% / 0% / 0%
D: sys3 added → 2 backends	136–184	100	2	189.5	100	4172	0	52% / 47% / 1%
E: sys4 added → 3 backends	192–260	100	3	285.2	102	2432	0	34% / 34% / 32%
F: 3 backends, 200 users	265–300	200	3	291.1	133	5710	0	34% / 33% / 33%

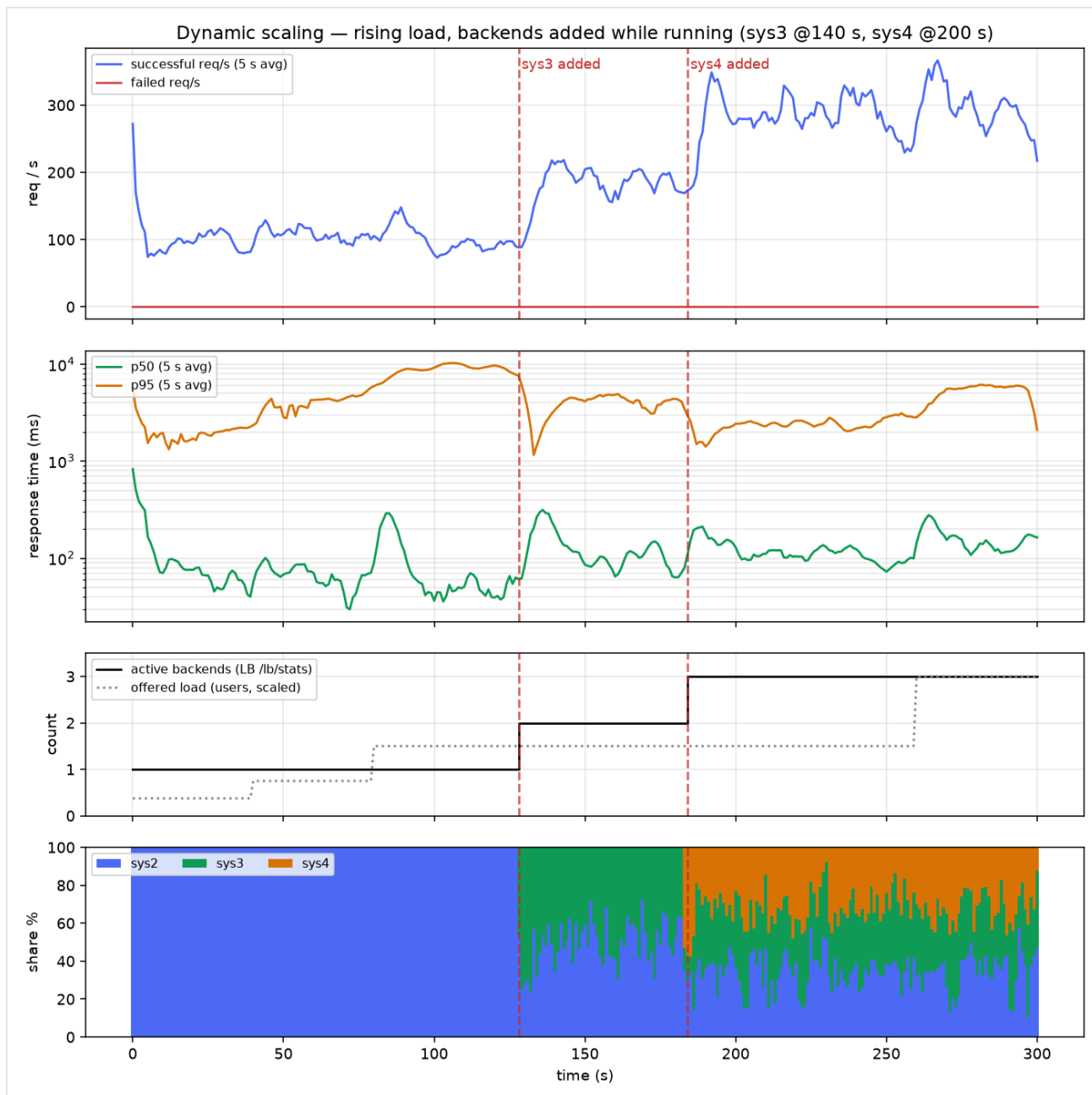


Figure 4 — Dynamic scaling timeline: throughput, response time, active backends and per-backend share; dashed lines = backend added.

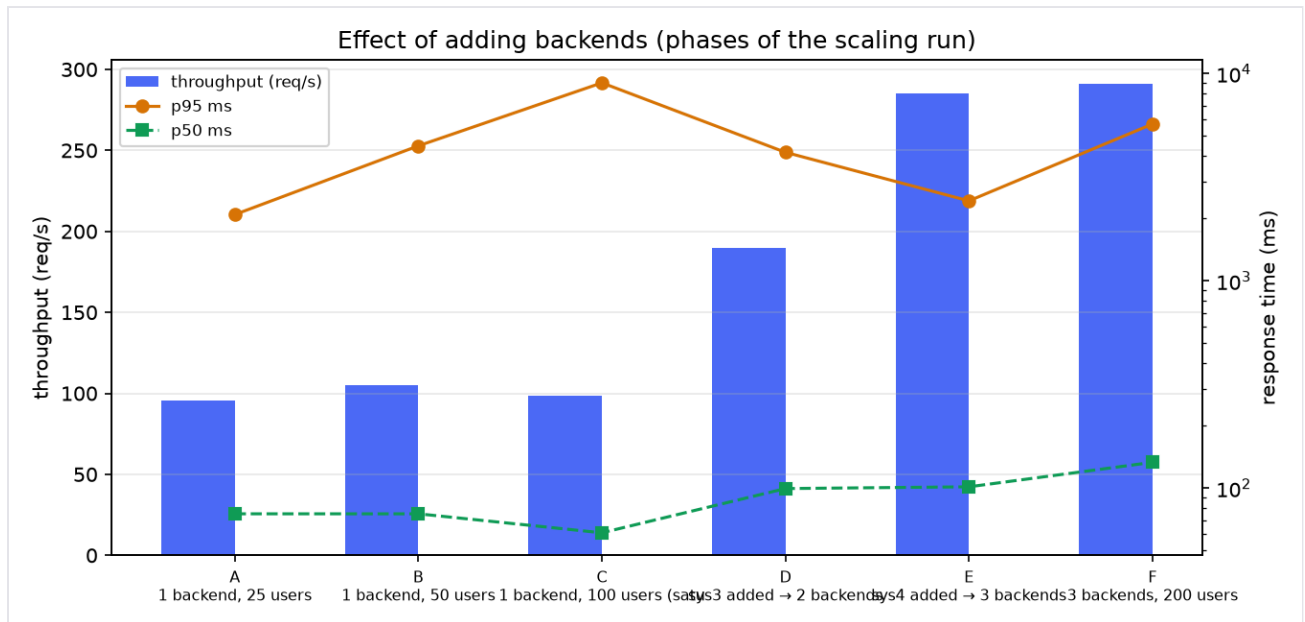


Figure 5 — Effect of adding each backend, per phase of the scaling run.

{{A_SCALE}}

9.2 Backend failure and recovery under load

Phase	Window (s)	Users	Active backends	Throughput (req/s)	p50 ms	p95 ms	Errors	Share sys2 / sys3 / sys4
healthy, 3 backends	5–29	100	3	280.4	125	2357	0	31% / 33% / 35%
sys3 killed → detection window	29–39	100	3	250.0	88	2737	36	34% / 26% / 39%
2 backends carry the load	39–90	100	2	184.1	94	4421	0	47% / 0% / 53%
sys3 restarted → re-admitted	90–100	100	3	225.0	182	1935	0	32% / 30% / 39%
3 backends again	100–150	100	3	279.8	96	2610	0	33% / 33% / 34%

Whole run: 34927 requests, 36 failed (0.103 %), 2 sends retried with the same id, 0 of those answered
duplicate:true (stored once).

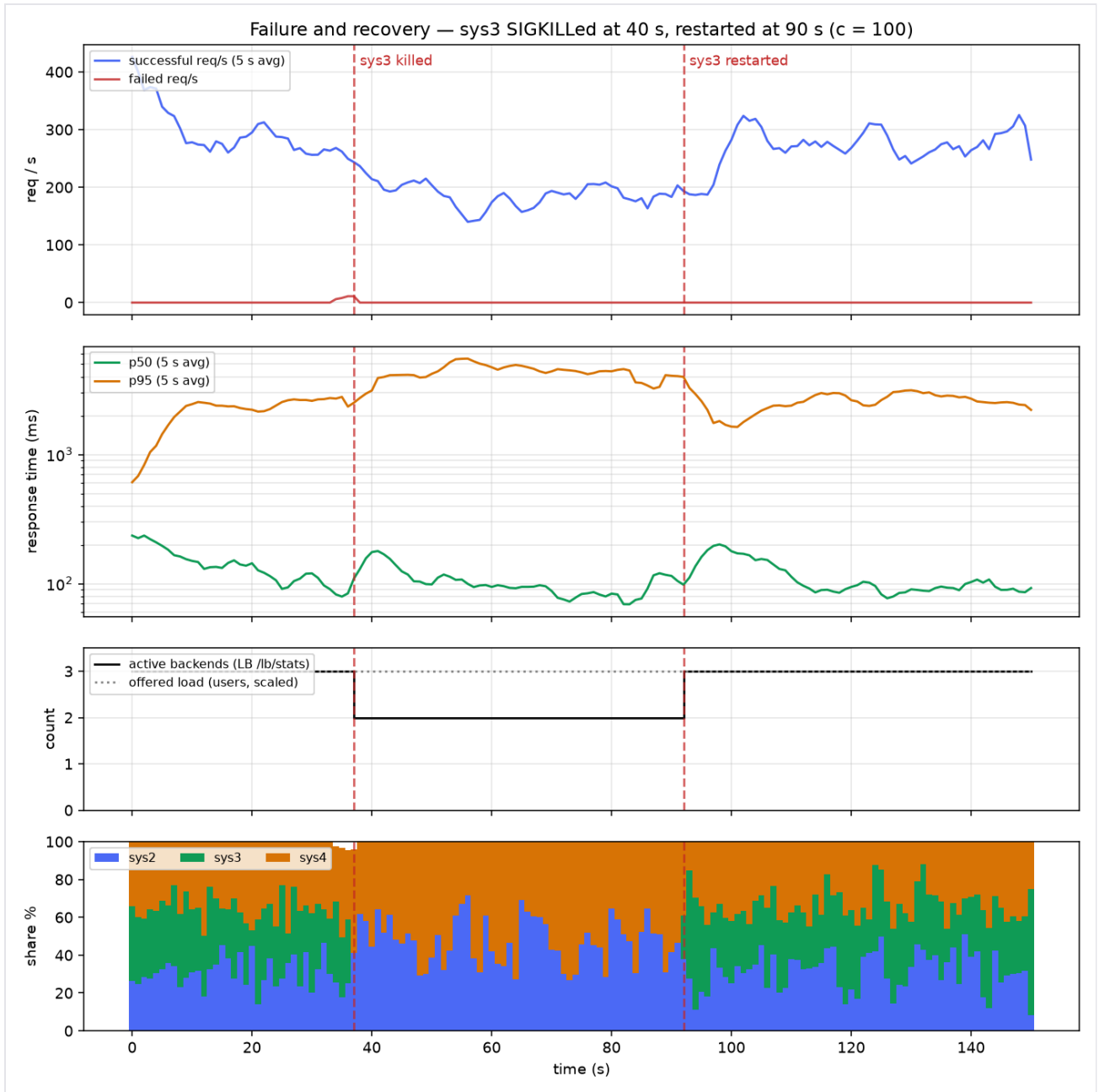


Figure 6 — Failure and recovery timeline (sys3 killed at 40 s, restarted at 90 s).

{{A_FAIL}}

9.3 Dynamic selection vs fixed rotation with one loaded backend

To show that selection really follows load, a CPU-burning process (`python3 -c 'while True: pass'`) was started on sys3 — on a 1-core container it takes roughly half of the core away from the backend. The same 50-user load was then run under each algorithm.

Algorithm	sys3 CPU hog	Throughput (req/s)	p50 ms	p95 ms	p99 ms	Errors	Share sys2 / sys3 / sys4
adaptive	yes	284.7	89.9	891.78	1313.49	0	32.6% / 34.1% / 33.3%
round_robin	yes	275.88	62.08	1042.86	2390.56	0	33.3% / 33.3% / 33.3%
least_connections	yes	288.33	80.64	1115.52	1466.85	0	32.7% / 33.3% / 34.0%
adaptive	no	287.51	95.37	663.96	1040.28	0	34.3% / 33.0% / 32.7%
round_robin	no	266.74	86.12	1047.37	1741.66	0	33.3% / 33.3% / 33.3%

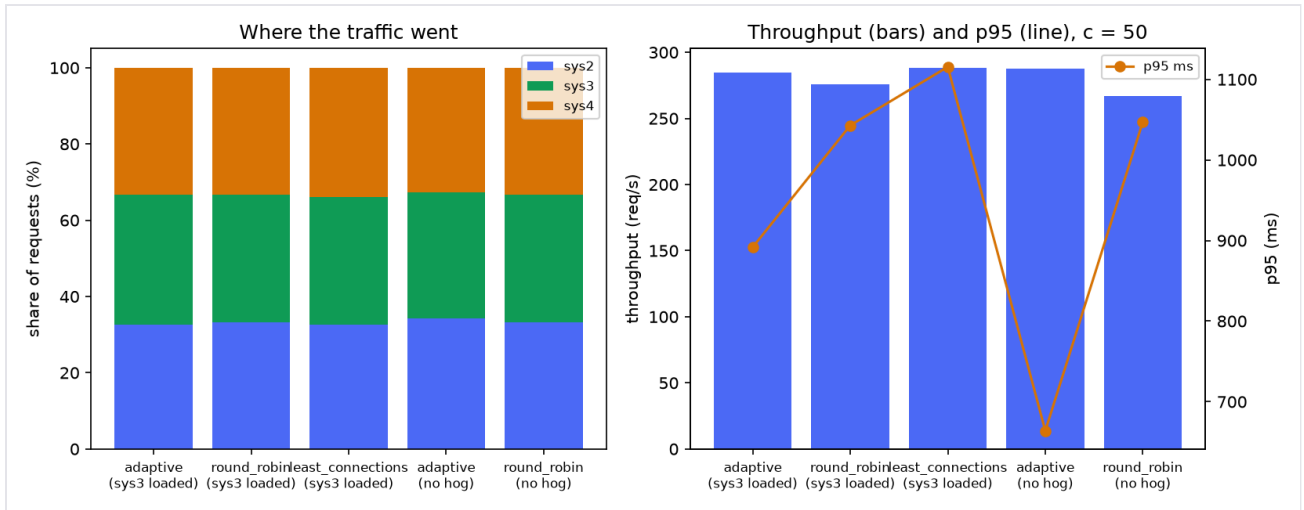


Figure 7 — Adaptive vs round robin vs least connections when sys3 is CPU-loaded.

{{A_ALGO}}

9.4 Comparison summary

Scenario	Configuration	Throughput (req/s)	p50 ms	p95 ms	Failed	Active backends
SCALE phase A: 1 backend, 25 users	1 backend(s)	95.5	75	2089	—	1
SCALE phase B: 1 backend, 50 users	1 backend(s)	105.2	75	4482	—	1
SCALE phase C: 1 backend, 100 users (saturated)	1 backend(s)	98.6	61	9028	—	1
SCALE phase D: sys3 added → 2 backends	2 backend(s)	189.5	100	4172	—	2
SCALE phase E: sys4 added → 3 backends	3 backend(s)	285.2	102	2432	—	3
SCALE phase F: 3 backends, 200 users	3 backend(s)	291.1	133	5710	—	3
ALGO: 50 users, sys3 CPU-loaded	adaptive	284.7	89.9	891.78	0	share sys3 = 34.1 %
ALGO: 50 users, sys3 CPU-loaded	round_robin	275.88	62.08	1042.86	0	share sys3 = 33.3 %
ALGO: 50 users, sys3 CPU-loaded	least_connections	288.33	80.64	1115.52	0	share sys3 = 33.3 %
FAIL: 100 users, sys3 killed + restarted	3 → 2 → 3 backends	240.63 (whole run)	101.89	2782.6	36 (0.103 %)	2–3

9.5 Persistence and duplicate prevention on the allotted systems

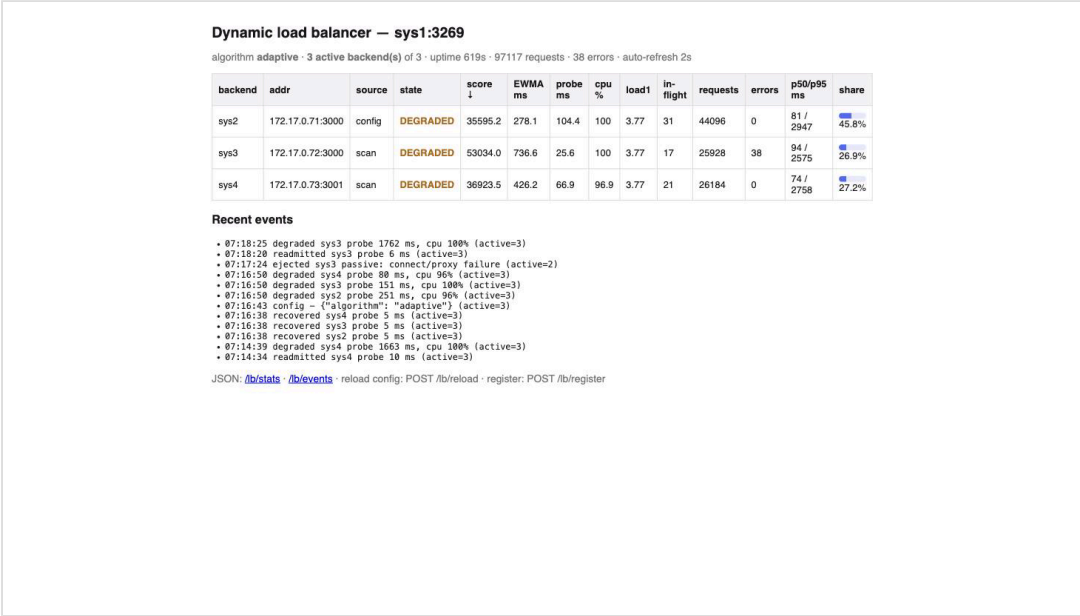
{{A_PERSIST}}

10. Analysis and Discussion

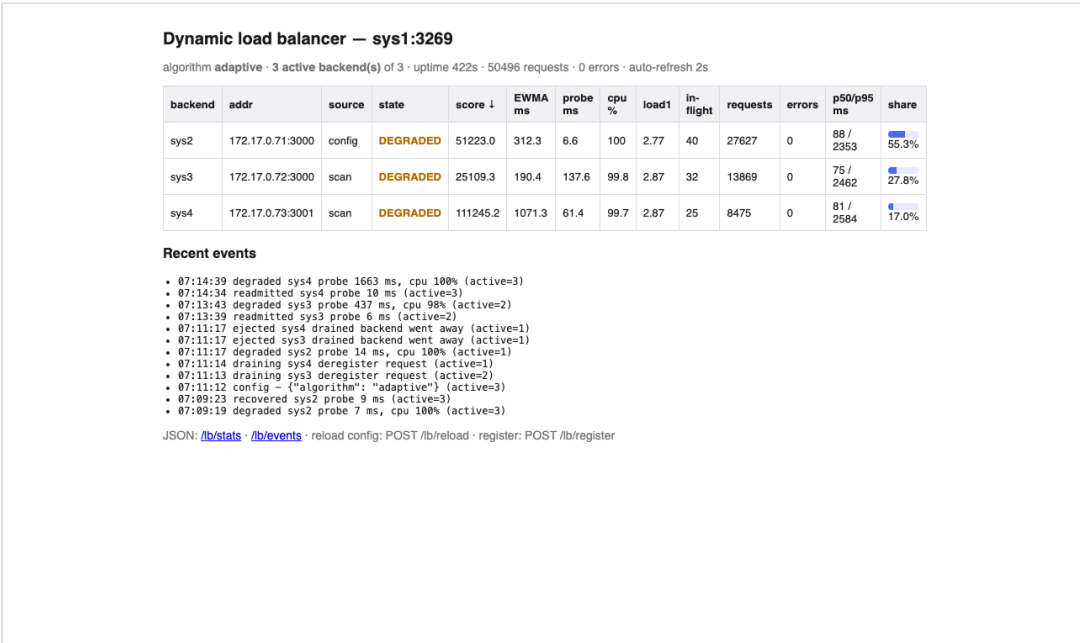
{{A_DISCUSSION}}

11. Screenshots and Evidence

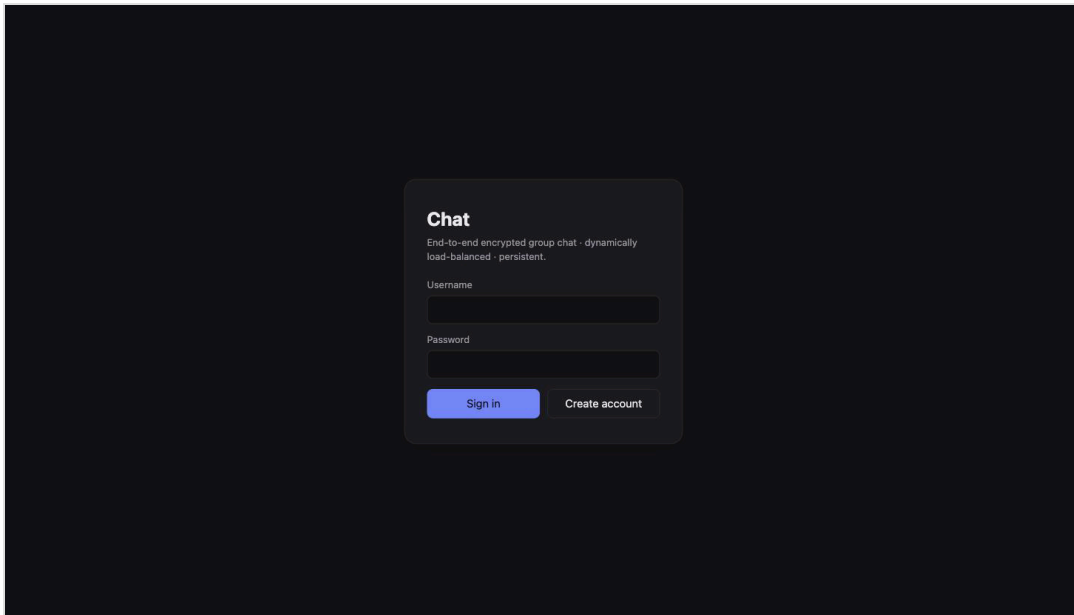
Live LB dashboard — three backends UP, adaptive scores, event log



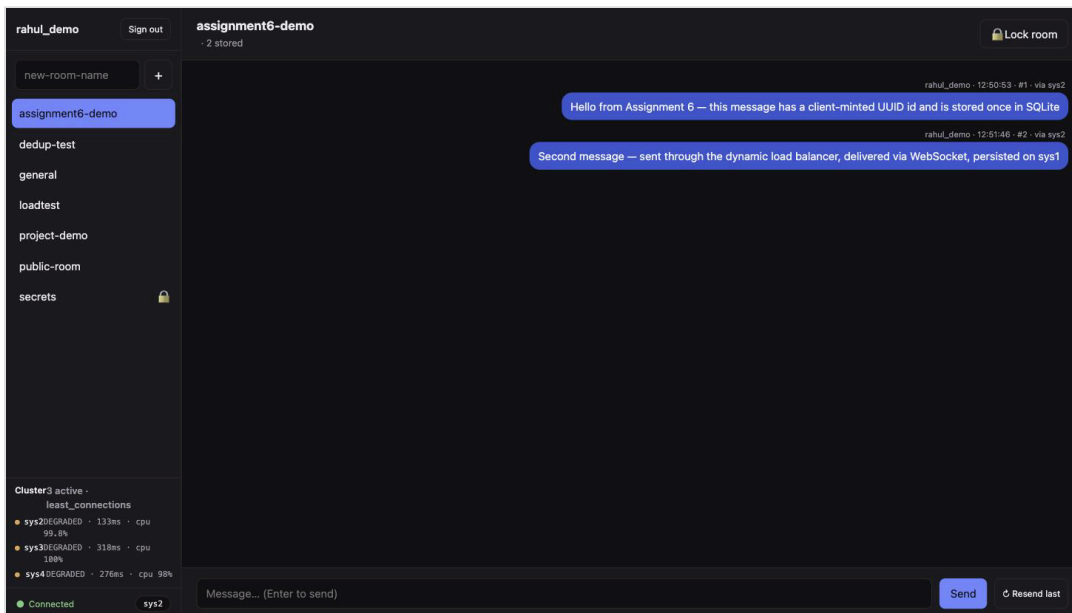
Dashboard during the scaling run — sys3 and sys4 admitted while load was running



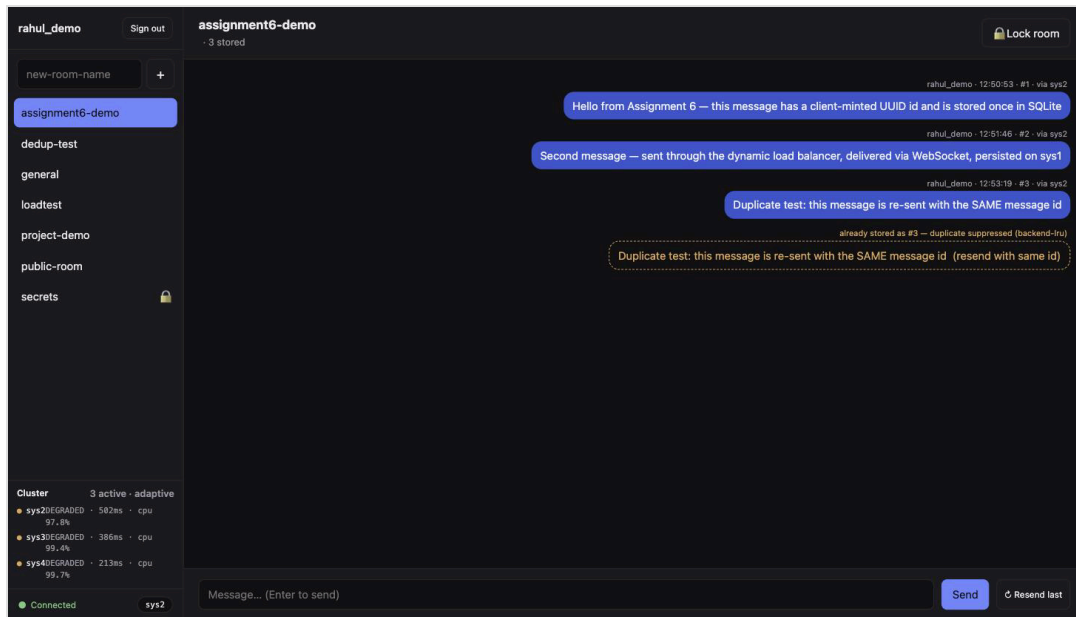
Chat login screen served through the load balancer



Conversation: message ids, seq numbers, serving backend, cluster panel



'Resend last' — same message id re-sent, server answers duplicate:true, nothing stored twice



Terminal captures

01_sanity_check.txt

```
$ bash scripts/sanity_check.sh
== Assignment 6 sanity check Fri Sep  4 12:53:56 IST 2026 ==
[✓] LB v2 (public)    http://10.1.75.53:3269/lb/health {"status":"ok","service":"lb","version":"v2-dynamic","active_backends":3}
[✓] app via LB        http://10.1.75.53:3269/health {"status":"ok","backend":"sys3","version":"v3-assignment6","uptime":342,"connections":1,"db_firehose":true,"load_algorithm=round_robin active_backends=3 total_requests=147670
  - sys2 172.17.0.71:3000 DEGRADED src=config ewma=452.9 score=25360.3 cpu=97.5 reqs=60463
  - sys3 172.17.0.72:3000 UP      src=scan ewma=52.3 score=209.0 cpu=85.2 reqs=42877
  - sys4 172.17.0.73:3001 UP      src=scan ewma=213.2 score=4477.5 cpu=92.9 reqs=42968
[✓] DB service sys1:5270 sqlite messages=154846 users=719 rooms=7 dups_rejected=4 subscribers=3
[✓] backend sys2 :3000 v3-assignment6 cpu 100 %
[✓] backend sys3 :3000 v3-assignment6 cpu 93.5 %
[✓] backend sys4 :3001 v3-assignment6 cpu 99.1 %
-- previous assignments --
[✓] Lab5 direct sys2 http://10.1.75.53:3270/health {"status":"ok","backend":"sys2","version":"v3-assignment6","uptime":948,"connections":0,"db_firehose":true,"load_algorithm=round_robin active_backends=3 total_requests=147670
[✓] Lab5 direct sys3 http://10.1.75.53:3271/health {"status":"ok","backend":"sys3","version":"v3-assignment6","uptime":343,"connections":1,"db_firehose":true,"load_algorithm=round_robin active_backends=3 total_requests=147670
[✓] Project Contour http://10.1.75.53:3272 (sys4:3000, untouched)
[✓] Lab5 state service sys1:5269 (still running, untouched)
```

02_dedup_test.txt

```
$ python3 scripts/dedup_test.py --url http://10.1.75.53:3269 --n 20
dedup test against http://10.1.75.53:3269, room dedup-test, N=20
T1 sent=20 stored=1 duplicate_responses=19 backends=['sys2', 'sys3', 'sys4'] PASS same id sequentially
T2 sent=20 stored=1 duplicate_responses=19 backends=['sys2', 'sys3', 'sys4'] PASS concurrent, 1 distinct seq
T3 sent=2 stored=1 duplicate_responses=1 backends=['sys2', 'sys4'] PASS seq 3 == 3
T4 sent=5 stored=1 duplicate_responses=4 backends=['sys2'] PASS Idempotency-Key header
T5 sent=20 stored=20 duplicate_responses=0 backends=['sys3'] PASS control: distinct ids
db: 112484 messages, 4 duplicates rejected in total (sqlite (node:sqlite, WAL))
```

03_db_stats_schema.txt

```
$ ssh lbsys1 'curl -s http://127.0.0.1:5270/stats'
{
  "service": "db",
  "engine": "sqlite (node:sqlite, WAL)",
  "file": "/home/student/assignment6/data/chat.sqlite",
  "db_bytes": 39800832,
  "users": 719,
  "rooms": 7,
  "messages": 156637,
  "duplicates_rejected_total": 4,
  "duplicates_rejected_since_boot": 4,
  "appended_since_boot": 45457,
  "requests": 236516,
  "subscribers": 3,
  "uptime": 970,
  "node": "v22.23.2"
}

$ ssh lbsys1 'ls -la ~/assignment6/data && python3 -c "import sqlite3; ...schema + counts..."'
total 42988
drwxrwxr-x 2 student student 4096 Sep  4 07:08 .
drwxrwxr-x 6 student student 4096 Sep  4 07:08 ..
-rw-r--r-- 1 student student 118 Sep  4 07:07 .migrated-from-lab5
-rw-r--r-- 1 student student 39800832 Sep  4 07:24 chat.sqlite
-rw-r--r-- 1 student student 32768 Sep  4 07:23 chat.sqlite-shm
-rw-r--r-- 1 student student 4173592 Sep  4 07:24 chat.sqlite-wal
CREATE TABLE dedup_log (
    -- audit trail of rejected duplicates
    n INTEGER PRIMARY KEY AUTOINCREMENT, id TEXT NOT NULL, room TEXT, via TEXT, at INTEGER NOT NULL);
CREATE TABLE messages (
    id TEXT PRIMARY KEY, -- globally unique message id (UUID v4): THE duplicate guard
    room TEXT NOT NULL,
    seq INTEGER NOT NULL, -- per-room sequence number, gap-free
    sender TEXT NOT NULL,
    ts INTEGER NOT NULL, -- client/backend timestamp (ms)
    kind TEXT NOT NULL, -- plain | enc | system
    text TEXT, -- plaintext rooms only
    env TEXT, -- E2E envelope JSON (ciphertext) for locked rooms
    via TEXT, -- backend that accepted it
    stored_at INTEGER NOT NULL, -- server receive time (ms)
    extra TEXT, -- optional JSON (e.g. roomMeta on system messages)
    UNIQUE(room, seq)
);
CREATE TABLE rooms (id TEXT PRIMARY KEY, locked INTEGER NOT NULL DEFAULT 0, epoch INTEGER NOT NULL DEFAULT 0,
    verifier TEXT, created INTEGER NOT NULL, created_by TEXT);
CREATE TABLE sessions (token TEXT PRIMARY KEY, username TEXT NOT NULL, created INTEGER NOT NULL, expires INTEGER NOT
NULL);
CREATE TABLE sqlite_sequence(name,seq);
CREATE TABLE users (username TEXT PRIMARY KEY, hash TEXT NOT NULL, created INTEGER NOT NULL);
CREATE INDEX idx_messages_room_seq ON messages(room, seq);
CREATE INDEX idx_sessions_expires ON sessions(expires);

rows: {'messages': 156642, 'users': 719, 'rooms': 7, 'dedup_log': 4}
duplicate ids in messages (must be 0): 0
recent dedup_log: [('014447af-4436-430c-8776-3027d0f0f681', 'dedup-test', 'sys4'), ('014447af-4436-430c-8776-
3027d0f0f681', 'dedup-test', 'sys4'), ('014447af-4436-430c-8776-3027d0f0f681', 'dedup-test', 'sys2')]
```

